

undercut 復元と 1本道 strict defeat 対応 実装計画書

目的

攻撃手法として **undercut** を戻し、**weak_negation / Ass** を Argument スキーマへ復元する。

ただし、今回の実装では argument tree を完全展開しない。1つの main argument に対して発生する議論は 1本道に制限し、最大で次の 4 argument までを扱う。

```
A <- B <- C <- D
```

- **A**: P の main argument
- **B**: O の defeating argument against A
- **C**: P の counterargument against B
- **D**: O の defeating argument against C

この制限により、文献 5章の **rebut / undercut / defeat / strict defeat / justified / overruled / defensible** を最小限扱いつつ、**justified** 判定の再帰的な多重ネストを避ける。

文献上の位置づけ

kido_kurihara.pdf では、5章で Prakken and Sartor [9] に基づく Argumentation Model が導入され、6章で negotiation application が示されている。

5章では次が定義されている。

- defeasible rule: $L1 \wedge \dots \wedge Lj \wedge \sim L_{j+1} \wedge \dots \wedge \sim L_{n-1} \Rightarrow Ln$
- $\sim L$: 「there is no evidence that L」を意味する弱否定
- **Ass(A)**: argument **A** に含まれる assumption
- **Conc(A)**: argument **A** に含まれる conclusion
- **rebut**
- **undercut**
- **defeat**
- **strict defeat**
- **justified**
- **overruled**
- **defensible**

一方、6章の具体例では各 agent の theory **Si** が only strict rules と明記されている。そのため、6章の例では **Ass(A)** が空になり、**undercut** は実質的に発生しない。

したがって、本計画は「6章の例をそのまま実装する」ものではない。6章の phase 構造を維持しながら、5章の full Argumentation Model のうち、**weak_negation / Ass / undercut / strict defeat** を導入する拡張である。

実装上の制限

文献の **justified** 定義は再帰的な探索を要求しうる。

A is justified if no argument defeats A
or if every argument which defeats A is also strictly defeated.

例えば **B justified?** を厳密に調べるには、**B** を defeat する **C**、さらに **C** を strictly defeat する **D**、さらにその先、という探索が発生しうる。

今回の実装では、この再帰を完全には追わない。次の制限を置く。

- **A** に対する **B** は 1つだけ生成する。
- **B** に対する **C** は 1つだけ生成する。
- **C** に対する **D** は 1つだけ生成する。
- **D** より先の argument は生成しない。
- **A** の状態は、この 1本道の範囲で **justified** / **overruled** / **defensible** に分類する。
- **overruled** が出了時点で、その candidate は不採用なので protocol 全体を停止する。
- **synthesis** に進むのは、両 agent の main argument がどちらも **defensible** で閉じた場合だけとする。

これは完全な argumentation semantics ではなく、6章の簡略 protocol に **undercut** と **strict defeat** を追加するための bounded dialogue protocol である。

復元対象

過去コミットでは、**d65cb02** [**feat**]rebutのみ、統合は合意核までで **undercut** と **weak_negation** の扱いが削られている。復元元としては、その親である **c5d0b0d** [**feat**]LangGraphを使った実装を参照する。

復元・変更する対象は次の通り。

- **src/agent/prompt.py**
 - **MAIN_ARGUMENT** に **antecedent.weak_negation** と **Ass** を戻す。
 - **DEFEATING_ARGUMENT** に **attack: rebut or undercut** を戻す。
 - **target** 情報を追加し、どの argument / field / statement を攻撃したかを出力させる。
 - **C / D** 生成用に、target argument を明示する prompt を追加または共通化する。
- **src/agent/schema/outputs/schema.py**
 - **ArgumentRecord** に **id, target_id, attack, status** を追加する。
 - **DefeatRelation** または同等の relation 記録 model を追加する。
- **src/agent/graph.py**
 - **A ← B ← C ← D** の 1本道 protocol に合わせて node / route を整理する。
 - **last_can_defeat** だけで分岐せず、**attack, target_id, status** を使う。
 - **extract_warrants** で **weak_negation** を再び抽出対象にする。
- **def.py**
 - **argument_statuses, defeat_relations, attack, target** を表示対象に含める。

用語と判定

rebut

X が **Y** を rebut するとは、**X.Conc** が **Y.Conc** と矛盾すること。

実装では LLM に `attack: "rebut"` と出力させるだけでなく、`target.statement` を持たせ、後段で検証する。

undercut

X が Y を undercut するとは、 $X.\text{Conc}$ が $Y.\text{Ass}$ を崩すこと。

$Y.\text{Ass}$ が空の場合、undercut は不可とする。

defeat

文献の定義に合わせる。

```
def defeats(x, y):
    if undercuts(x, y):
        return True
    if rebuts(x, y) and not undercuts(y, x):
        return True
    return False
```

今回の 1 本道 protocol では、以下のように運用する。

- B が A を undercut するなら、 $B \text{ defeats } A$ は成立する。
- B が A を rebut する場合、 P/A が B を undercut できるなら $B \text{ defeats } A$ は成立しない。
- C と D についても同じ rule で判定する。

strict defeat

```
def strictly_defeats(x, y):
    return defeats(x, y) and not defeats(y, x)
```

ただし今回の protocol では、 $C \text{ strictly defeats } B$ の確認を、 $D \text{ defeats } C$ が成立するかどうかで近似する。

- D が出ない、または $D \text{ defeats } C$ が成立しない場合、 $C \text{ strictly defeats } B$ とみなす。
- $D \text{ defeats } C$ が成立する場合、 $C \text{ strictly defeats } B$ は不成立とする。

この近似は、 $B \text{ does not defeat } C$ を直接厳密判定する代わりに、 O が C に対する有効な defeating argument D を出せるかで判定する設計である。

状態分類

A justified

次のいずれかの場合、 $A \text{ justified}$ とする。

- O が A に対する有効な B を出せない。
- B が rebut だが、 P が B を undercut できるため、 $B \text{ defeats } A$ が成立しない。

- **B defeats A** は成立したが、**C strictly defeats B** が成立する。

A overruled

次の場合、**A overruled** とする。

- **B defeats A** が成立する。
- P が **B** を defeat する **C** を出せない。
- または **C** が出ても **C defeats B** が成立しない。

この場合、bounded protocol 上では **B justified** とみなし、**A** は justified argument に defeat されたものとして扱う。

overruled は不採用の確定状態である。今回の system protocol では、片方の main argument が **overruled** になった時点で議論全体を終了し、P/O 交代や synthesis には進まない。

A defensible

次の場合、**A defensible** とする。

- **B defeats A** が成立する。
- **C defeats B** も成立する。
- しかし **D defeats C** も成立するため、**C strictly defeats B** が成立しない。

つまり、**A** は攻撃されたが、**B** も完全には通らず、**C** も **B** を完全に退けられていない未決着状態である。

defensible は未決着状態である。片方の main argument が **defensible** の場合だけ P/O を交代して相手側の main argument を試す。両方の main argument が **defensible** になった場合に限り、synthesis に進む。

アクティビティ図

```

flowchart TD
    Start([Start thread]) --> PMain[P: main argument A]
    PMain --> ODefeat[O: defeating argument B against A]

    ODefeat --> BExists{B is generated?}
    BExists -- No --> AJustified1[A justified]
    BExists -- Yes --> BAttack[B attack type]

    BAttack -- rebut --> PUndercutB{Can P undercut B?}
    PUndercutB -- Yes --> AJustified2[A justified]
    BAttack -- undercut --> BDefeatsA[B defeats A]

    BDefeatsA --> PCounter[P: counterargument C against B]
    PCounter --> CExists{C is generated?}
    CExists -- No --> AOverruled1[A overruled]
    CExists -- Yes --> CAttack[C attack type]

    AJustified1 --> BJustified[B justified in bounded protocol]
    AJustified2 --> BJustified
    AOverruled1 --> BJustified
    CAttack --> BJustified
  
```

```

    CAttack -- rebut --> OUndercutC{Can O undercut C?}
    OUndercutC -- Yes --> A0verruled2[A overruled
C rebut defeat fails]
    OUndercutC -- No --> CDefeatsB[C defeats B]

    CAttack -- undercut --> CDefeatsB

    CDefeatsB --> OCounter[O: defeating argument D against C]
    OCounter --> DExists{D is generated?}
    DExists -- No --> AJustified3[A justified
C strictly defeats B]
    DExists -- Yes --> DAttack{D attack type}

    DAttack -- rebut --> PUndercutD{Can P undercut D?}
    PUndercutD -- Yes --> AJustified4[A justified
D rebut defeat fails]
    PUndercutD -- No --> ADefensible1[A defensible
D defeats C]

    DAttack -- undercut --> ADefensible2[A defensible
D defeats C]

    AJustified1 --> Finish([Finish])
    AJustified2 --> Finish
    AJustified3 --> Finish
    AJustified4 --> Finish

    A0verruled1 --> RejectedEnd([End: rejected])
    A0verruled2 --> RejectedEnd
    ADefensible1 --> Switch
    ADefensible2 --> Switch

    Switch --> TriedBoth{Both agents tried main as defensible?}
    TriedBoth -- No --> PMain
    TriedBoth -- Yes --> Synthesis[Synthesis]
    Synthesis --> End([End])

```

抽象化した組み立て方

上のアクティビティ図は実装時に分かりやすい具体フローである。一方、実装では同じ判定が複数回出てくるため、次の部品に分けて使い回す。

再利用する部品

generate_attack(attacker, target, purpose)

agent が **target** に対する argument を生成する。

用途:

- **O** が **A** に対して **B** を生成する。
- **P** が **B** に対して **C** を生成する。

- **O** が **C** に対して **D** を生成する。

返す値:

```
AttackResult(
  argument=ArgumentRecord | None,
  can_attack=bool,
  attack="rebut" | "undercut" | None,
  target_id=str,
)
```

can_undercut(agent, target)

target が **rebut** argument として使われる場合に、相手がその **target.Ass** を崩せるかを見る。

用途:

- **B.attack == "rebut"** のとき、**P** が **B** を undercut できるか。
- **C.attack == "rebut"** のとき、**O** が **C** を undercut できるか。
- **D.attack == "rebut"** のとき、**P** が **D** を undercut できるか。

返す値:

```
UndercutCheck(
  can_undercut=bool,
  argument=ArgumentRecord | None,
)
```

resolve_defeat(attacker, target, defender)

attacker が **target** を defeat するかを判定する共通部品。

処理:

```
attacker が存在しない:
-> no_defeat

attacker.attack == undercut:
-> defeat

attacker.attack == rebut:
  defender が attacker を undercut できる:
    -> no_defeat
  defender が attacker を undercut できない:
    -> defeat
```

これにより、**B defeats A**, **C defeats B**, **D defeats C** を同じ関数で扱える。

返す値:

```
DefeatCheck(
    defeats=bool,
    blocked_by_undercut=bool,
    blocker=ArgumentRecord | None,
)
```

bounded_defense(main, defeater, proponent, opponent)

A と **B** が与えられた後、**C** と **D** まで見て **A** の status を返す。

処理:

```
P が B に対する C を生成する
C が B を defeat しない:
-> A overruled

C が B を defeat する:
O が C に対する D を生成する
D が C を defeatしない:
-> A justified

D が C を defeatする:
-> A defensible
```

返す値:

```
ThreadResult(
    main_status="justified" | "overruled" | "defensible",
    arguments=list[ArgumentRecord],
    relations=list[DefeatRelation],
)
```

main_status == "overruled" は protocol の停止を意味する。**main_status == "defensible"** の場合だけ、相手側 main argument へ交代する。

抽象化版アクティビティ図

```
flowchart TD
    Start([Start thread]) --> Main[P: generate main A]
    Main --> AttackB[O: generate_attack against A -> B]
    AttackB --> ResolveBA[resolve_defeat B against A]
```

```
ResolveBA --> BAResult{B defeats A?}
BAResult -- No --> Justified[A justified]
BAResult -- Yes --> Defense[bounded_defense A against B]

Defense --> DefenseResult{bounded defense result}
DefenseResult -- B strictly defeated --> Justified
DefenseResult -- B justified --> Overruled[A overruled]
DefenseResult -- unresolved --> Defensible[A defensible]

Justified --> Finish([Finish])
Overruled --> RejectedEnd([End: rejected])
Defensible --> Switch

Switch --> TriedBoth{Both agents tried main as defensible?}
TriedBoth -- No --> Main
TriedBoth -- Yes --> Synthesis[Synthesis]
Synthesis --> End([End])
```

部品と具体フローの対応

具体フロー	抽象部品
0: defeating argument B against A	generate_attack(0, A, "defeat_main")
B attack type	resolve_defeat(B, A, P)
Can P undercut B?	can_undercut(P, B) inside resolve_defeat
P: counterargument C against B	generate_attack(P, B, "defend_main") inside bounded_defense
Can 0 undercut C?	can_undercut(0, C) inside resolve_defeat(C, B, 0)
0: defeating argument D against C	generate_attack(0, C, "test_strict_defeat") inside bounded_defense
Can P undercut D?	can_undercut(P, D) inside resolve_defeat(D, C, P)

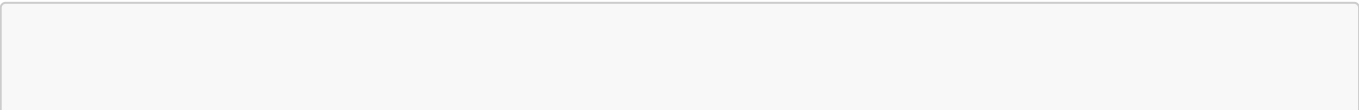
実装方針

LangGraph の node は具体フローに合わせて分けてもよいが、内部実装は上の部品を使い回す。

推奨:

- node 名は具体的にしてデバッグしやすくする。
- helper は抽象化して重複を避ける。
- routing は ThreadResult.main_status だけを見る。

例:




```

async def o_defeat_a(state: State) -> dict[str, Any]:
    result = await generate_attack(
        state=state,
        attacker=state.current_opponent,
        target_id=state.current_main_id,
        purpose="defeat_main",
    )
    defeat = await resolve_defeat(
        state=state,
        attacker=result.argument,
        target=state.current_argument,
        defender=state.current_proponent,
    )
    ...

```

LangGraph node 設計

ノードは agent の行動単位を基本にする。

p_main

現在の P が main argument A を生成する。

出力:

- current_main_id
- current_argument
- history
- dialogue_history

o_defeat_a

現在の O が A に対する defeating argument B を生成する。

出力:

- b_argument
- b_attack
- b_target_id
- b_can_defeat

p_undercut_b

B.attack == "rebut" の場合だけ実行する。

P が B を undercut できるかを判定し、可能なら undercut argument を生成する。

出力:

- p_can_undercut_b
- optional c_or_undercut_argument

ここで undercut できた場合、**B defeats A** は成立しないため、**A justified** で終了する。

p_counter_b

B defeats A が成立した後、P が **B** に対する counterargument **C** を生成する。

出力:

- c_argument
- c_attack
- c_target_id
- c_can_defeat

C が出ない、または **C defeats B** が成立しない場合、bounded protocol では **B justified** とみなし、**A overruled** とする。

o_defeat_c

C defeats B が成立した後、O が **C** に対する defeating argument **D** を生成する。

出力:

- d_argument
- d_attack
- d_target_id
- d_can_defeat

D defeats C が成立しない場合、**C strictly defeats B** とみなし、**A justified** とする。

D defeats C が成立する場合、**C strictly defeats B** は不成立とし、**A defensible** とする。

route_after_thread

A justified / **A overruled** / **A defensible** で thread が閉じたときに実行する。

- **A justified** の場合、成功として終了する。
- **A overruled** の場合、不採用確定として終了する。
- **A defensible** かつまだ片方の main argument しか試していない場合、P/O を交代して **p_main** へ戻る。
- 両者の main argument がどちらも **defensible** で閉じた場合だけ、**synthesis** へ進む。

synthesis

両者の main argument がどちらも **defensible** で閉じた場合に実行する。

既存の **extract_warrants** → **generalize** → **integrate** を使う。ただし、warrant には **weak_negation** も含める。

追加する状態

State に次を追加する。

```

current_proponent: AgentName
current_opponent: AgentName
main_attempts: list[ArgumentRecord]
thread_statuses: dict[str, str]
defeat_relations: list[DefeatRelation]
current_main_id: str | None
b_argument_id: str | None
c_argument_id: str | None
d_argument_id: str | None

```

ArgumentRecord は次の optional field を持つ。

```

id: str
target_id: str | None
attack: Literal["rebut", "undercut"] | None
status: Literal["justified", "overruled", "defensible", "undetermined"] | None

```

DefeatRelation は次の形にする。

```

class DefeatRelation(BaseModel):
    attacker_id: str
    target_id: str
    attack: Literal["rebut", "undercut"]
    valid: bool
    reason: str | None = None

```

プロンプト改修

MAIN_ARGUMENT

weak_negation と **Ass** を復元する。

必須フィールド:

- **rules[*].antecedent.strong**
- **rules[*].antecedent.weak_negation**
- **rules[*].consequent**
- **Conc**
- **Ass**

strict rule の場合、**weak_negation** と **Ass** は空配列でよい。

DEFEATING_ARGUMENT

rebut と **undercut** の両方を許可する。

出力には **target** を必ず含める。

```
{
  "can_defeat": "YES or NO",
  "Argument": {
    "attack": "rebut or undercut",
    "target": {
      "argument_id": "target argument id",
      "field": "Conc or Ass",
      "statement": "target statement"
    },
    "rules": [],
    "Conc": [],
    "Ass": []
  }
}
```

UNDERCUT_CHECK

B.attack == "rebut" または **D.attack == "rebut"** のときに使う。

目的:

- target argument の **Ass** を崩せるか判定する。
- 可能なら undercut argument を生成する。

出力:

```
{
  "can_undercut": "YES or NO",
  "Argument": {
    "attack": "undercut",
    "target": {
      "argument_id": "target argument id",
      "field": "Ass",
      "statement": "target assumption"
    },
    "rules": [],
    "Conc": [],
    "Ass": []
  }
}
```

VALIDATE_ATTACK

LLM の **attack** ラベルをそのまま信じないため、軽量 validator を追加する。

入力:

- attacker argument
- target argument
- declared attack
- declared target

出力:

```
{
  "valid": true,
  "attack": "rebut or undercut",
  "reason": "short reason"
}
```

統合フェーズへの影響

`extract_warrants` では、main argument の最後の rule から次を抽出する。

```
{
  "warrant": {
    "antecedent": {
      "strong": [],
      "weak_negation": []
    },
    "consequent": ""
  }
}
```

`weak_negation` を統合フェーズで無視しない。特に、main argument が undercut で崩れた場合、どの assumption が崩れたかを synthesis の材料として保持する。

実装順序

1. `schema.py` に `id`, `target_id`, `attack`, `status`, `DefeatRelation` を追加する。
2. `prompt.py` に `weak_negation` / `Ass` / `undercut` / `target` を復元・追加する。
3. `_extract_json_from_argument` 周辺に `attack`, `target`, `Conc`, `Ass`, rule-level `weak_negation` の抽出 helper を追加する。
4. `validate_attack` helper を追加する。
5. `p_main`, `o_defeat_a`, `p_undercut_b`, `p_counter_b`, `o_defeat_c`, `route_after_thread` を `graph.py` に実装する。
6. 現行の `ag1_main`, `ag2_attack_ag1`, `ag1_attack_ag2` などを新 protocol に合わせて置き換える、または wrapper 化する。
7. `extract_warrants` に `weak_negation` を戻す。
8. `def.py` の表示処理に `attack`, `target`, `status`, `defeat_relations` を追加する。
9. unit test を追加する。
10. integration test で AG1/AG2 の main が両方 `defensible` で閉じた後に synthesis へ進むことを確認する。

テスト計画

LLM を使わない unit test:

- **Ass** が抽出できる。
- rule-level **weak_negation** が flatten される。
- **undercut** は target **Ass** が空なら不可。
- **rebut** は target **Conc** への矛盾として扱われる。
- **B.rebut** に対して P が **B** を undercut できる場合、**B defeats A** は成立しない。
- **B.undercut** の場合、**B defeats A** が成立する。
- **C defeats B** かつ **D defeats C** がない場合、**A justified** になる。
- **C defeats B** かつ **D defeats C** がある場合、**A defensible** になる。
- **B defeats A** かつ **C** がない場合、**A overruled** になる。

integration test:

- AG1 main が rebut されるが、AG1 が undercut して justified になる。
- AG1 main が undercut され、AG1 が counter できず overruled になる。
- AG1 main が undercut され、AG1 が counter するが AG2 が **D** を出して defensible になる。
- AG1 thread と AG2 thread がどちらも **defensible** で閉じたら synthesis へ進む。
- どちらかの thread が **overruled** になったら synthesis へ進まず終了する。

注意点

この計画は、文献 5章の argumentation semantics を完全実装するものではない。

特に、**justified** の完全な再帰評価や、複数 defeater を持つ argument tree は扱わない。今回扱うのは、6章の negotiation protocol に合わせた bounded one-chain protocol である。

その代わり、次の性質を明示的に実装する。

- strict rule では **Ass** は空でよい。
- defeasible rule では **weak_negation** / **Ass** を持てる。
- **undercut** は **Ass** に対する攻撃として扱う。
- **rebut** は相手から **undercut** されると defeat にならない。
- **strict defeat** は **C** と **D** の bounded check で扱う。
- **overruled** / **defensible** は、この bounded check の範囲での状態である。